

***** chap-3 Practise book solved programs *****

QB:-90:- Write a program to handle `ArrayIndexOutOfBoundsException` using `try-catch-finally` and `throw`.

```
import java.util.Scanner;

class HandleAIOBsException {
    public static void main(String[] args) {
        int[] numbers = { 1, 2, 3, 4, 5 };
        Scanner sc = new Scanner(System.in);
        try {
            int index = sc.nextInt(); // Accessing an index that is out of bounds
            if (index < 0 || index >= numbers.length) {
                throw new ArrayIndexOutOfBoundsException("Index " + index + " is out of bounds.");
            }
        } catch (ArrayIndexOutOfBoundsException e) {
            System.err.println(e.getMessage());
        } finally {
            System.out.println("Program execution completed.");
        }
    }
}
```

QB:-91:-Write a program to handle `InterruptedException`, `IllegalArgumentException` using `try-catch-finally` and `throw`

```
public class HandleExceptions {
    public static void main(String[] args) {
        try {
            // Simulating InterruptedException
            simulateInterruptedException();

            // Simulating IllegalArgumentException
            simulateIllegalArgumentException(-5);
        } catch (InterruptedException e) {
            // Handling InterruptedException
            System.err.println("InterruptedException occurred: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            // Handling IllegalArgumentException
            System.err.println("IllegalArgumentException occurred: " + e.getMessage());
        } finally {
            // Finally block executes whether an exception occurred or not
            System.out.println("Program execution completed.");
        }
    }

    public static void simulateInterruptedException() throws InterruptedException {
        try {
            // Simulating a task that may be interrupted
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            // Re-throwing InterruptedException
            throw e;
        }
    }
}
```

```

53     }
54 }
55
56 public static void simulateIllegalArgumentException(int value) {
57     if (value < 0) {
58         // Throwing IllegalArgumentException if the value is negative
59         throw new IllegalArgumentException("Value cannot be negative: " + value);
60     }
61 }
62 }

```

QB:-92:- Write a method for computing x^y by doing repetitive multiplication. x and y are of type integer and are to be given as command line arguments. Raise and handle exception(s) for invalid values of x and y . Also define method main.

```

68 public class Calculator {
69
70     public static void power(int x, int y) {
71
72         int result = 1;
73         for (int i = 0; i < y; i++) {
74             result *= x;
75         }
76         System.out.println(x + "^" + y + " = " + result);
77     }
78     public static void main(String[] args) {
79         try {
80             int x = Integer.parseInt(args[0]);
81             int y = Integer.parseInt(args[1]);
82             if (y < 0) {
83                 throw new IllegalArgumentException();
84             }
85             power(x, y);
86         } catch (ArrayIndexOutOfBoundsException e) {
87             System.out.println("Error: Must Enter 2 arguments only");
88         } catch (NumberFormatException e) {
89             System.out.println("Error: Enter integer input only");
90         } catch (IllegalArgumentException e) {
91             System.out.println("Error: You cannot Write negative value for Y ");
92         }
93     }
94 }

```

QB:- 93:-Write an application that searches through its command-line argument. If an argument is found that does not begin with and upper case letter, display error message and terminate.

```

100 public class ArgumentChecker {
101
102     public static void checkArguments(String[] args) {
103         for (String arg : args) {

```

```

104         if (!Character.isUpperCase(arg.charAt(0))) {
105             throw new IllegalArgumentException("Invalid argument: " + arg);
106         }
107     }
108
109 }
110
111 public static void main(String[] args) {
112     try {
113         if (args.length == 0) {
114             System.out.println("Enter atleast one Arguments");
115         } else {
116             checkArguments(args);
117             System.out.println("All arguments are valid.");
118         }
119     } catch (IllegalArgumentException e) {
120         System.out.println(e.getMessage());
121     }
122 }
123
124
125

```

QB:-94:- Write a program to create user define exception MyException. Define a class ExceptionDemo, that has a method named compute(int a) which throws a MyException object

```

128 when compute(int a)'s integer parameter is greater than 10
129 import java.util.Scanner;
130
131 class MyException extends RuntimeException {
132
133 }
134 class ExceptionDemo {
135     public void compute(int a) throws MyException {
136         if (a > 10) {
137             throw new MyException();
138         } else {
139             System.out.println("Integer parameter is less than or equal to 10");
140         }
141     }
142 }
143 public class Main {
144     public static void main(String[] args) {
145         ExceptionDemo d = new ExceptionDemo();
146         try {
147             Scanner sc = new Scanner(System.in);
148             System.out.println("Enter any Integer Number: ");
149             int a = sc.nextInt();
150             d.compute(a);
151         } catch (MyException e) {
152             System.out.println("Integer parameter is greater than 10");
153         }
154     }
155 }
156
157

```

158 QB:-95:- Write a program to handle NullPointerException.

```
159
160 class HandleNullPointerException {
161     public static void main(String[] args) {
162         String str = null;
163
164         try {
165             int length = str.length();
166             System.out.println("Length of the string: " + length);
167         } catch (NullPointerException e) {
168             System.err.println(e.getMessage());
169         }
170     }
171 }
```

173 QB:-96:- Write a program to create user define exception MyException. Define a class
174 ExceptionDemo that has a method named compute() which throws a MyException object, when
175 compute()'s integer parameter is divisible by 7 and not divisible by 5.

```
176 import java.util.Scanner;
177
178 class MyException extends RuntimeException {
179     public MyException() {
180         super("Number is divisible by 7 but not divisible by 5");
181     }
182 }
183
184 public class ExceptionDemo {
185     public void compute(int num) throws MyException {
186         if (num % 7 == 0 && num % 5 != 0) {
187             throw new MyException();
188         }
189         System.out.println("Number is valid");
190     }
191
192     public static void main(String[] args) {
193         ExceptionDemo d = new ExceptionDemo();
194         try {
195             Scanner sc = new Scanner(System.in);
196             System.out.print("Enter any Integer Number: ");
197             int a = sc.nextInt();
198             d.compute(a);
199         } catch (MyException e) {
200             System.out.println("Error: " + e.getMessage());
201         }
202     }
203 }
204 /*PS F:\Exception\LJ> javac ExceptionDemo.java
205 PS F:\Exception\LJ> java ExceptionDemo
206 Enter any Integer Number: 21
207 Error: Number is divisible by 7 but not divisible by 5
208 PS F:\Exception\LJ> java ExceptionDemo
209 Enter any Integer Number: 35
210 Number is valid */
211
212
```

```

213 QB:-97:-Write an application that searches through its command-line argument. If first
    command
214 line argument is found zero then display error message.
215 public class QB97 {
216     public static void main(String[] args) {
217         if (args.length > 0) {
218             try {
219                 int num = Integer.parseInt(args[0]);
220                 if (num == 0) {
221                     throw new IllegalArgumentException("First argument cannot be zero");
222                 }
223             } catch (NumberFormatException e) {
224                 System.out.println("Error: First argument must be an integer");
225                 return;
226             } catch (IllegalArgumentException e) {
227                 System.out.println("Error: " + e.getMessage());
228                 return;
229             }
230         } else {
231             System.out.println("Error: No arguments provided");
232             return;
233         }
234         System.out.println("All arguments are valid");
235     }
236 }
237 /*PS F:\Exception\LJ> javac QB97.java
238 PS F:\Exception\LJ> java QB97
239 Error: No arguments provided
240 PS F:\Exception\LJ> java QB97 one
241 Error: First argument must be an integer
242 PS F:\Exception\LJ> java QB97 12
243 All arguments are valid
244 PS F:\Exception\LJ> java QB97 012
245 All arguments are valid
246 PS F:\Exception\LJ> java QB97 0 123
247 Error: First argument cannot be zero */
248
249 QB:-98:-Write an application that searches through its command-line argument. If sum of
    first and
250 second command line argument is found 10 then display error message.
251 public class QB98 {
252     public static void main(String[] args) {
253         if (args.length >= 2) {
254             try {
255                 int num1 = Integer.parseInt(args[0]);
256                 int num2 = Integer.parseInt(args[1]);
257                 if (num1 + num2 == 10) {
258                     throw new IllegalArgumentException("Sum of first two arguments
259                         cannot be 10");
260                 }
261             } catch (NumberFormatException e) {
262                 System.out.println("Error: Arguments must be integers");
263                 return;
264             } catch (IllegalArgumentException e) {
265                 System.out.println("Error: " + e.getMessage());

```

```

265         return;
266     }
267 } else {
268     System.out.println("Error: At least two arguments must be provided");
269     return;
270 }
271 System.out.println("All arguments are valid");
272 }
273 }
274 /*PS F:\Exception\LJ> javac QB98.java
275 PS F:\Exception\LJ> java QB98
276 Error: At least two arguments must be provided
277 PS F:\Exception\LJ> java QB98 one 10
278 Error: Arguments must be integers
279 PS F:\Exception\LJ> java QB98 1 5
280 All arguments are valid
281 PS F:\Exception\LJ> java QB98 1 9
282 Error: Sum of first two arguments cannot be 10
283 PS F:\Exception\LJ> java QB98 0 10
284 Error: Sum of first two arguments cannot be 10 */
285
286 QB:-99:- write a program to enter the five subject marks out of 100 of student. If any
subject
marks is less than 35 then program will generate exception
287
288 import java.util.Scanner;
289
290 class SubjectMarksException extends Exception {
291     public SubjectMarksException(String message) {
292         super(message);
293     }
294 }
295
296 public class QB99 {
297     public static void main(String[] args) {
298         Scanner sc = new Scanner(System.in);
299         int[] marks = new int[5];
300         try {
301             for (int i = 0; i < 5; i++) {
302                 System.out.print("Enter marks for subject " + (i + 1) + ": ");
303                 marks[i] = sc.nextInt();
304                 if (marks[i] < 35) {
305                     throw new SubjectMarksException("Marks cannot be less than 35");
306                 }
307             }
308             System.out.println("All marks are valid");
309         } catch (SubjectMarksException e) {
310             System.out.println("Exception: " + e.getMessage());
311         }
312     }
313 }
314
315 /*PS F:\Exception\LJ> javac QB99.java
316 PS F:\Exception\LJ> java QB99
317 Enter marks for subject 1: 40
318 Enter marks for subject 2: 36
319 Enter marks for subject 3: 65

```

```

319 Enter marks for subject 4: 41
320 Enter marks for subject 5: 45
321 All marks are valid
322 PS F:\Exception\LJ> java QB99
323 Enter marks for subject 1: 10
324 Exception: Marks cannot be less than 35 */
325
326 QB:-100:- It is required to maintain and process the status of total 9 resources. The
status value is to
327 be stored in an integer array of dimension 3x3. The valid status of a resource can be one
of
328 the 2followings:free: indicated by integer value 0 occupied: indicated by integer value 1
329 inaccessible: indicated by integer value 2
330 Declare a class called ResourcesStatus, having data member called statusRef, referring to
a
331 two dimensional array (3x3) of integers to be used to refer to the above mentioned status
values.
332
333 Define a member method called processStausCount that counts and displays total number
334 of free resources, total number of occupied resources and total number of inaccessible
335 resources.
336 The exception to be raised and handled if total number of occupied resources exceeds
total
337 number of free resources. The handler marks status of all inaccessible resources as free.
338 Accept initial status values from command line arguments and initialize the array. Raise
and
339 handle user defined exception if invalid status value given
340
341 class InvalidStatusException extends RuntimeException {
342     InvalidStatusException(String message) {
343         super(message);
344     }
345 }
346
347 class ResourcesStatus {
348     int[][] statusRef;
349
350     ResourcesStatus(int[] initialStatus) {
351         if (initialStatus.length != 9) {
352             throw new IllegalArgumentException("Initial status array must have 9
elements");
353         }
354
355         this.statusRef = new int[3][3];
356         for (int i = 0; i < 3; i++) {
357             for (int j = 0; j < 3; j++) {
358                 int status = initialStatus[i * 3 + j];
359                 if (status < 0 || status > 2) {
360                     throw new InvalidStatusException("Invalid status value at position ("
+ i + ", " + j + ")");
361                 }
362                 this.statusRef[i][j] = status;
363             }
364         }
365     }
366

```

```

367 void processStatusCount() {
368     int freeCount = 0;
369     int occupiedCount = 0;
370     int inaccessibleCount = 0;
371
372     // Count status types
373     for (int i = 0; i < 3; i++) {
374         for (int j = 0; j < 3; j++) {
375             if (statusRef[i][j] == 0) {
376                 freeCount++;
377             } else if (statusRef[i][j] == 1) {
378                 occupiedCount++;
379             } else if (statusRef[i][j] == 2) {
380                 inaccessibleCount++;
381             }
382         }
383     }
384
385     // Display counts
386     System.out.println("Total number of free resources: " + freeCount);
387     System.out.println("Total number of occupied resources: " + occupiedCount);
388     System.out.println("Total number of inaccessible resources: " + inaccessibleCount
389 );
390
391     // Handle exception
392     if (occupiedCount > freeCount) {
393         System.out.println("Exception: Total number of occupied resources exceeds
394 total number of free resources");
395         markInaccessibleAsFree();
396     }
397
398 void markInaccessibleAsFree() {
399     // Mark all inaccessible resources as free
400     for (int i = 0; i < 3; i++) {
401         for (int j = 0; j < 3; j++) {
402             if (statusRef[i][j] == 2) {
403                 statusRef[i][j] = 0;
404             }
405         }
406     }
407
408 public static void main(String[] args) throws Exception {
409     try {
410         int[] initialStatus = new int[args.length];
411         for (int i = 0; i < args.length; i++) {
412             initialStatus[i] = Integer.parseInt(args[i]);
413         }
414         ResourcesStatus resourcesStatus = new ResourcesStatus(initialStatus);
415         resourcesStatus.processStatusCount();
416     } catch (NumberFormatException e) {
417         System.out.println("Error: Invalid status value");
418     } catch (InvalidStatusException e) {
419         System.out.println("Error: " + e.getMessage());

```

```

420     }
421 }
422 }
423
424
425 QB:-101:-Write a complete program to accept N integer numbers from the command line.
Raise and
426 handle exceptions for following cases :
427 - when a number is -ve
428 - when a number is evenly divisible by 10
429 - when a number is greater than 1000 and less than 2000
430 - when a number is greater than 7000
431 Skip the number if an exception is raised for it, otherwise add it to find total sum
432
433 public class NumberValidator {
434     public static void main(String[] args) {
435         int sum = 0;
436         for (int i = 0; i < args.length; i++) {
437             try {
438                 int num = Integer.parseInt(args[i]);
439                 if (num < 0) {
440                     throw new IllegalArgumentException("Negative number not allowed");
441                 }
442                 if (num % 10 == 0) {
443                     throw new Exception(num+" Number is evenly divisible by 10");
444                 }
445                 if (num > 1000 && num < 2000) {
446                     throw new Exception(num+" Number is greater than 1000 and less than
2000");
447                 }
448                 if (num > 7000) {
449                     throw new Exception(num+" Number is greater than 7000");
450                 }
451                 sum += num;
452             } catch (NumberFormatException e) {
453                 System.out.println(args[i] + " is not a valid integer");
454             } catch (IllegalArgumentException e) {
455                 System.out.println(e.getMessage());
456             } catch (Exception e) {
457                 System.out.println(e.getMessage() + " - Skipping number " + args[i]);
458             }
459         }
460         System.out.println("Total sum is: " + sum);
461     }
462 }
463
464
465
466 QB:-102:-Declare a class called coordinate to represent 3 dimensional Cartesian
coordinates( x, y and z).
467 Define following methods:
468 - constructor
469 - display, to print values of members
470 - add_coordinates, to add three such coordinate objects to produce a resultant
471 coordinate object. Generate and handle exception if x, y and z coordinates of the

```

resultare zero.

- main, to show use of above methods.

```
public class Coordinate {
    double x;
    double y;
    double z;

    public Coordinate(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    public void display() {
        System.out.println("(" + x + ", " + y + ", " + z + ")");
    }

    public static Coordinate addCoordinates(Coordinate c1, Coordinate c2, Coordinate c3)
        throws Exception {
        double x = c1.x + c2.x + c3.x;
        double y = c1.y + c2.y + c3.y;
        double z = c1.z + c2.z + c3.z;
        if (x == 0 && y == 0 && z == 0) {
            throw new Exception("Resultant coordinate has zero values for x, y and z");
        }
        return new Coordinate(x, y, z);
    }

    public static void main(String[] args) {
        Coordinate c1 = new Coordinate(1, 2, 3);
        Coordinate c2 = new Coordinate(4, 5, 6);
        Coordinate c3 = new Coordinate(7, 8, 9);
        c1.display();
        c2.display();
        c3.display();
        try {
            Coordinate result = Coordinate.addCoordinates(c1, c2, c3);
            result.display();
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

/*when we will put all the values 0 Exception occurs*/

QB:-103:-Write an application that searches through its command-line argument. If any of command

line argument is found negative then display error message.

```
public class QB96 {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Enter atleast one Argument: ");
        } else {
            for (String arg : args) {
                try {
```

```
524         int num = Integer.parseInt(arg);
525         if (num < 0) {
526             throw new IllegalArgumentException("Negative number found");
527         }
528     } catch (NumberFormatException e) {
529         // Ignore non-integer arguments
530     } catch (IllegalArgumentException e) {
531         System.out.println("Error: " + e.getMessage());
532         return;
533     }
534 }
535 System.out.println("All arguments are valid");
536 }
537
538 }
539 }
540 /*PS F:\Exception\LJ> javac QB96.java
541 PS F:\Exception\LJ> java QB96
542 Enter atleast one Argument:
543 PS F:\Exception\LJ> java QB96 12 1 one
544 All arguments are valid
545 PS F:\Exception\LJ> java QB96 12 5 -66
546 Error: Negative number found */
547
548
```